Planned Implementation of EnCase Forensic and Volatility 3

1. Purpose

This document outlines how EnCase Forensic and Volatility 3 would be incorporated into the Digital Forensics and Evidence Analysis portion of the project.

The purpose of implementing these tools is to expand the project's forensic capabilities beyond filesystem examination and deleted-file recovery. EnCase would provide an additional commercial forensic examination platform for acquiring and analyzing digital evidence. Volatility 3 would introduce memory forensics, allowing the examination of volatile system data such as running processes, network connections, loaded modules, and potentially suspicious activity.

These tools are proposed additions. Their implementation and testing must be completed and documented before they can be reported as tools successfully used in the project.

2. EnCase Forensic Implementation Plan

2.1 Tool Overview

EnCase Forensic, developed by OpenText, is a commercial digital forensic investigation platform. It supports evidence acquisition, forensic examination, file analysis, recovery, and reporting.

EnCase would complement the existing Autopsy and Sleuth Kit workflow by providing another platform for examining forensic images and documenting findings.

Planned uses:

- Create and manage a forensic investigation case.
- Acquire or import digital evidence.
- Examine filesystems and file metadata.
- Identify deleted files and recover available data.
- Search for relevant keywords and file types.
- Examine timestamps and other file attributes.
- Generate forensic examination reports.
- Validate evidence integrity through hash verification.

2.2 Installation and Environment

EnCase is proprietary software and requires an appropriate license or authorized evaluation/training access. Before installation, the supported operating systems and hardware requirements must be checked against the current OpenText documentation.

The project currently uses an Apple M4 Mac and a Parrot Security OS ARM64 virtual machine. EnCase compatibility with this environment must not be assumed. The preferred implementation would be a supported Windows workstation or compatible Windows virtual machine, subject to OpenText's current requirements.

Planned installation procedure:

1. Obtain EnCase Forensic through an authorized OpenText license, educational program, or evaluation arrangement.
2. Review the official installation and hardware requirements.
3. Prepare a compatible Windows environment.
4. Install EnCase using the authorized installer.
5. Activate the software using the appropriate license.
6. Record the installed version and environment details.
7. Capture screenshots showing the successful installation and application launch.

2.3 Test Case and Evidence

The existing Digital Evidence Analysis Report: Case 001 provides a suitable starting point for a controlled comparison.

The case used a 256 MB ext4 forensic disk image containing sample documents, log files, and deliberately deleted files. The original image was preserved, a working copy was created, and the image integrity was checked using SHA-256 and MD5 hashes.

For an EnCase examination:

1. Create a separate working copy of the existing forensic image.
2. Preserve the original image as read-only.
3. Record the image's filename, size, acquisition details, and SHA-256 hash.
4. Create a new EnCase case named `Digital_Evidence_Analysis_Case001_EnCase`.
5. Add the working forensic image as an evidence item, using a supported image-import method.
6. Confirm that the image is recognized and that its filesystem can be examined.
7. Run the available evidence-processing and analysis functions.
8. Compare the results with the existing Autopsy and Sleuth Kit findings.

The existing image should be used only if EnCase supports its format and filesystem for the intended examination. If it cannot be examined reliably, create a separate test image in a supported format and document the difference.

2.4 Planned Examination Procedures

The examination would focus on the following areas:

Examination area	Planned activity	Expected evidence
Filesystem analysis	Examine directories, files, and metadata	File listings and metadata
Deleted-file examination	Search for deleted or recoverable files	Deleted-file records and recovered files
Keyword searching	Search for known sample text	Search results and file locations
File identification	Examine file types and signatures	File-type identification results
Timeline analysis	Review available timestamps	Chronological activity information
Hash verification	Calculate and compare hashes	Evidence integrity records
Reporting	Export examination results	EnCase examination report

2.5 Validation and Comparison

EnCase results would be compared with the findings already documented in Case 001.

Examples include:

- Confirming whether the same three live files are identified.
- Checking whether the two deleted-file entries are detected.
- Comparing recoverable PDF content with the files recovered by Foremost and PhotoRec.
- Comparing file metadata and timestamps with the existing Sleuth Kit and Autopsy timeline.
- Comparing SHA-256 hashes of recovered files where the same data is recovered.

Any differences must be documented rather than automatically treated as errors. Different forensic tools may interpret filesystem structures and deleted-file metadata differently.

2.6 Evidence to Collect

The following evidence should be retained after implementation:

- EnCase version and installation screenshots.
- Screenshot of the created forensic case.
- Screenshot showing the imported evidence image.

- Screenshots of filesystem and deleted-file examination.
- Keyword search results.
- Timeline or metadata examination results.
- Hash verification results.
- Exported EnCase report.
- Comparison of EnCase results against Autopsy and Sleuth Kit.

3. Volatility 3 Implementation Plan

3.1 Tool Overview

Volatility 3 is an open-source memory forensics framework. Unlike EnCase and Autopsy, which are commonly used to examine disk evidence, Volatility focuses on analyzing memory images.

Memory forensics is important because certain evidence may exist only while a system is running. Examples include active processes, network connections, loaded modules, command history, and suspicious code residing in memory.

Volatility would expand the project by adding a method for examining volatile evidence that cannot necessarily be recovered from a disk image.

Planned uses:

- Identify running and terminated process artifacts where available.
- Examine process relationships and parent-child relationships.
- Identify network connections and related artifacts.
- Examine loaded modules and drivers.
- Search for suspicious process memory.
- Review command-line information and available command history.
- Identify potentially suspicious memory regions.
- Export and preserve analysis results.

3.2 Installation on Parrot OS

Volatility 3 is Python-based and can be installed in the existing Parrot OS ARM64 virtual machine, provided its Python environment meets the requirements.

The official project documentation identifies Python 3.8 or later as a requirement.

Planned installation procedure:

1. Start the Parrot OS virtual machine.
2. Open a terminal.
3. Create a dedicated working directory.
4. Create and activate a Python virtual environment.

5. Install Volatility 3.
6. Verify that the `vol` command is available.
7. Record the installed version.

Example commands:

```
None
mkdir -p ~/forensics/volatility
cd ~/forensics/volatility

python3 -m venv venv
source venv/bin/activate

python -m pip install --upgrade pip
python -m pip install volatility3

vol -h
```

The installation should be performed in a separate environment so that it does not interfere with the existing forensic tools or system Python packages.

3.3 Memory Image Acquisition

Volatility analyzes memory images; it does not acquire memory itself. A separate memory acquisition utility or a trusted forensic memory sample is therefore required.

For this project, the preferred approach would be to use a controlled virtual machine and collect a memory image while it is running.

Possible approaches include:

- Acquire memory from an authorized Linux test system using a compatible acquisition utility such as AVML.
- Use a suitable Windows memory sample from a reputable forensic training dataset.
- Use a controlled virtual machine to generate known activity and then acquire its memory for examination.

The acquisition method must be compatible with the operating system, kernel, and virtual machine environment. The acquisition utility, version, command, date, time, and system details should be recorded.

Important: A disk image such as `forensic_image.dd` is not a substitute for a RAM image. Volatility must be provided with a suitable memory capture.

3.4 Memory Evidence Preservation

Once a memory image has been acquired:

1. Save the original capture in a dedicated evidence directory.
2. Record its filename, size, source system, and acquisition timestamp.
3. Calculate its SHA-256 hash.
4. Preserve the original capture as read-only.
5. Create a working copy for examination.
6. Calculate the working copy's SHA-256 hash.
7. Confirm that the original and working-copy hashes match.
8. Conduct all Volatility analysis against the working copy.

Suggested directory structure:

```
None
forensics/
└── volatility/
    ├── evidence/
    │   ├── original_memory.raw
    │   └── working_memory.raw
    ├── reports/
    │   ├── system_info.txt
    │   ├── processes.txt
    │   ├── process_tree.txt
    │   ├── network_connections.txt
    │   └── suspicious_memory.txt
    ├── logs/
    └── hashes/
        └── memory_hashes.txt
```

The actual memory-image extension will depend on the acquisition tool and format.

3.5 Initial Memory Analysis

The first examination step would be to determine whether Volatility recognizes the memory image and can identify the operating system information.

For a Windows memory image:

```
None
vol -f evidence/working_memory.raw windows.info
```

This helps identify information about the operating system represented by the image.

The following analysis commands would then be used where supported by the memory image:

Process listing

```
None
vol -f evidence/working_memory.raw windows.pslist
```

Purpose: Identify processes represented in memory.

Process tree

```
None
vol -f evidence/working_memory.raw windows.pstree
```

Purpose: Examine parent-child process relationships and identify unusual process execution patterns.

Command-line information

```
None
vol -f evidence/working_memory.raw windows.cmdline
```

Purpose: Review command-line arguments associated with processes.

Network information

```
None
vol -f evidence/working_memory.raw windows.netscan
```

Purpose: Identify network artifacts available in the memory image, including connection information.

Loaded modules

```
None
vol -f evidence/working_memory.raw windows.dlllist
```

Purpose: Examine DLLs associated with processes.

Memory analysis

```
None
vol -f evidence/working_memory.raw windows.malfind
```

Purpose: Identify memory regions with characteristics that may warrant further investigation.

These commands are examples for Windows memory images. The correct plugins depend on the operating system and the available artifacts. A plugin producing a result does not, by itself, prove malicious activity.

3.6 Linux Memory Analysis

If a Linux memory image is used instead, Volatility provides Linux-specific plugins.

Examples include:

```
None
vol -f evidence/working_memory.raw linux.pslist
vol -f evidence/working_memory.raw linux.pstree
vol -f evidence/working_memory.raw linux.bash
vol -f evidence/working_memory.raw linux.lsmod
vol -f evidence/working_memory.raw linux.kmsg
```

These can help examine processes, process relationships, available Bash history artifacts, loaded kernel modules, and kernel messages.

Linux analysis may require a compatible symbol table for the captured kernel. The kernel version and symbol-table compatibility should be checked before interpreting results. If the necessary symbols are unavailable, the limitation must be documented rather than treating missing output as proof that no evidence exists.

3.7 Validation and Reporting

Volatility findings would be evaluated using multiple artifacts rather than relying on one plugin result.

For example:

- Compare process listings with process-tree relationships.
- Examine command-line arguments in the context of the associated process.
- Correlate network artifacts with process information where possible.
- Investigate unusual modules or memory regions using additional evidence.
- Record any missing, inaccessible, or unsupported artifacts.

Each finding should include:

- Finding ID.
- Memory image name.
- Plugin or command used.
- Relevant process, PID, address, or artifact.
- Output filename.
- Description of the observation.
- Interpretation and confidence level.
- Limitations and supporting evidence.

The results should be exported to text or other appropriate output formats and included in the forensic case documentation.

4. Integration into the Existing Forensic Case

EnCase and Volatility would be added to the existing evidence-management process rather than operated as independent, undocumented exercises.

The existing Case 001 report already documents Autopsy, Sleuth Kit, Foremost, PhotoRec, image hashes, recovered files, timelines, and evidence-register entries.

The new tools would contribute additional examination records.

Tool	Evidence type	Role in the investigation
Autopsy	Disk image	Filesystem examination, deleted-file review, and timeline analysis
Sleuth Kit	Disk image	Filesystem metadata, inode examination, and timeline reconstruction
Foremost	Disk image	File carving and recovery
PhotoRec	Disk image	File recovery through carving

EnCase	Disk image or other supported evidence	Independent forensic examination and reporting
Volatility 3	RAM image	Memory analysis and volatile artifact examination

Evidence Register Additions

The evidence register should be extended with entries for the new tool outputs.

Suggested entries:

Evidence ID	Evidence description	Evidence type
EV-012	EnCase case and examination output	Forensic examination records
EV-013	EnCase exported report	Forensic report
EV-014	Volatility original memory capture	Memory evidence
EV-015	Volatility working memory image	Working evidence copy
EV-016	Volatility process analysis output	Memory analysis
EV-017	Volatility network analysis output	Memory analysis
EV-018	Volatility suspicious-memory analysis output	Memory analysis

These are proposed IDs and should only be assigned when the corresponding evidence has actually been created and entered into the register.

All evidence should include a timestamp, investigator, source, description, filename, SHA-256 hash where applicable, validation status, and notes. Chain-of-custody records should document every transfer or handling event.

5. Validation and Success Criteria

The implementation would be considered successful when the following requirements are demonstrated.

Requirement	Validation method
EnCase installed	Application launches and version is recorded

EnCase case created	Case creation screenshot and case files
EnCase evidence imported	Evidence item visible in the case
EnCase examination completed	Examination output and report
EnCase results compared	Comparison with Autopsy and Sleuth Kit
Volatility installed	Version and command-line help output
Memory image acquired	Acquisition record and original memory file
Memory image integrity verified	Matching original and working-copy hashes
Volatility analysis completed	Successful execution of applicable plugins
Findings documented	Saved output and interpretation
Evidence preserved	Evidence register and chain-of-custody records

6. Expected Benefits to the Project

Implementing these tools would improve the forensic project in several ways.

EnCase

- Adds experience with a commercial forensic examination platform.
- Provides an independent examination method for disk evidence.
- Expands the project's evidence-processing and reporting capabilities.
- Allows comparison of findings across multiple forensic applications.

Volatility

- Adds memory forensics to the existing filesystem-focused investigation.
- Provides access to volatile artifacts that may not exist in disk evidence.
- Supports process and network activity analysis.
- Expands the investigation's ability to identify potentially suspicious system activity.

Together, the tools would provide a broader forensic workflow covering both persistent storage and volatile memory.

7. Limitations and Considerations

- EnCase is commercial software, and licensing or authorized evaluation access may be required.
- EnCase installation must use an environment supported by OpenText. Compatibility with the project's Apple M4 and Parrot ARM64 environment must be verified.
- Volatility requires a valid memory image and does not perform memory acquisition itself.
- Memory acquisition can alter the live system being examined. The acquisition method and its potential impact must be documented.
- Volatility results depend on memory-image quality, operating-system compatibility, and available symbols.
- A missing artifact does not necessarily mean that the activity did not occur.
- Tool output must be interpreted and corroborated with other evidence.
- The use of multiple forensic tools does not automatically establish legal admissibility. Proper evidence handling, documentation, validation, and applicable legal procedures remain necessary.

8. Implementation Priority

The recommended order is:

1. **Volatility 3:** Install and test in the existing Parrot OS environment. It is open source and provides a practical way to add a new forensic capability without purchasing a commercial license.
2. **EnCase:** Investigate educational or evaluation licensing, verify supported Windows requirements, and prepare a compatible examination environment.
3. **Cross-tool validation:** Compare EnCase results with the existing Autopsy and Sleuth Kit case.
4. **Documentation:** Add screenshots, version information, commands, output files, hashes, and findings to the forensic report and evidence register.

9. Conclusion

EnCase and Volatility 3 are proposed additions to the project's forensic toolkit.

EnCase would provide another platform for disk-based forensic examination, while Volatility would introduce memory analysis and the examination of volatile system artifacts.

Their implementation would build on the existing Case 001 investigation and expand the project's forensic coverage. Once installed, tested, and supported by examination evidence, these tools would also help satisfy the rubric's requirement to demonstrate the use of multiple recognized forensic tools.

Until those implementation and validation steps are completed, EnCase and Volatility should be classified as planned tools rather than completed project components.